Group Algorithm Engineering Assignment 1

CMP_SC 4050

2026 October 12

Connor Souders, Carson Mensink, Alexis Paten, Chris Crowder

Deniz Kavzak Ufuktepe

Algorithm Descriptions

We implemented the three required rules, Earliest Finish, Earliest Start, and Shortest Duration, along with one rule of our own, Longest Duration. All four algorithms follow the same rule. Each job is a half-open interval $[s, f)$, so a job that finishes at time t is compatible with a job that starts at time t. An algorithm sorts the jobs by its own priority and then considers them in that order. A job is selected when it does not overlap any job that was already selected. Ties in the sort are broken by the smaller original index, so every algorithm is deterministic and every experiment is reproducible.

The compatibility check is shared by all four algorithms. It sorts the distinct endpoints once and keeps two Fenwick trees over them, one counting the starts and one counting the finishes of the selected jobs. Each check takes $O(\log n)$ time, so checking all n jobs takes $O(n \log n)$ time, the same order as the sort that precedes it.

Earliest Finish

Earliest Finish sorts the jobs by finish time in increasing order and breaks ties by original index. Finishing as early as possible leaves the largest remaining time window for the jobs that follow. A standard exchange argument shows that this algorithm always produces an optimal schedule. Its running time is $O(n \log n)$, because the sort and the compatibility check each take $O(n \log n)$ time.

Earliest Start

Earliest Start sorts the jobs by start time in increasing order, with ties broken by original index. The idea is to start work as soon as possible, but it ignores how long a job runs. If the first job starts early and runs for a long time, it can block many shorter jobs that come after it. Its running time is $O(n \log n)$.

Shortest Duration

Shortest Duration sorts the jobs by duration, $f_j - s_j$, in increasing order, with ties broken by original index. The idea is that short jobs use little time and leave room for others. The rule fails when a short job sits across the boundary between two other jobs that are compatible with each other, because selecting the short job blocks both of them. Its running time is $O(n \log n)$.

Longest Duration (Our Group's Heuristic)

Longest Duration is our group's heuristic for Part 1D. It sorts the jobs by duration in decreasing order, with ties broken by original index. The idea is to commit to the largest jobs first and fit smaller jobs into the gaps around them, which tests the opposite intuition to Shortest

Duration. It is a poor heuristic because long jobs overlap many other jobs, so selecting one often blocks several jobs at once. Its running time is O(n log n).

Counterexamples

A counterexample is an input on which a greedy algorithm returns fewer jobs than the optimum. Before searching, we checked that our framework detects the two known failures given in the assignment (Table 1). The counterexamples reported after that were all found by the automated search in Part 3 (seed 4050) and confirmed by the exact solver. In every table, jobs are lettered in input order, so A is the first job, and each selection is listed in time order.

Required Sanity Tests

Table 1. Results of the two required sanity tests.

Test	Input Intervals	Greedy Selection	Optimal Selection	Sizes	Result
Sanity Test 1 (Earliest Start)	A: (0, 10) B: (1, 2) C: (2, 3) D: (3, 4) E: (4, 5)	A	B, C, D, E	1 vs 4	Passed
Sanity Test 2 (Shortest Duration)	A: (2, 4) B: (0, 3) C: (3, 6)	A	B, C	1 vs 2	Passed

In Sanity Test 1, Earliest Start picks A because it starts at time 0, and A covers the entire range [0, 10), so no other job can be selected. In Sanity Test 2, Shortest Duration picks A because it is the shortest job, but A overlaps both B and C, which are compatible with each other. Both tests produced the expected sizes, so the framework detects these known failures.

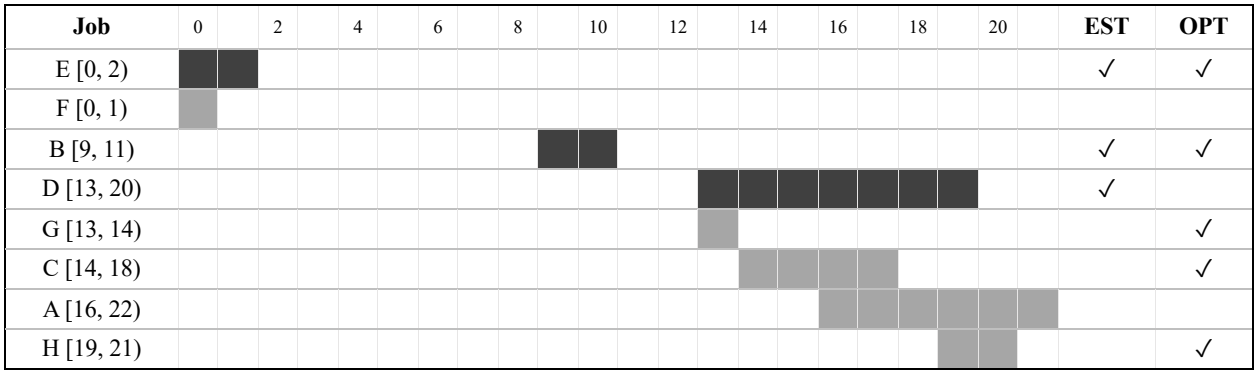
Earliest Start

Table 2. Earliest Start counterexample (found by the Part 3 search after 4 attempts).

Input Intervals	Algorithm Selection	Greedy Size	Optimal Selection	OPT Size
A: (16, 22) B: (9, 11) C: (14, 18) D: (13, 20) E: (0, 2) F: (0, 1) G: (13, 14) H: (19, 21)	E, B, D	3	E, B, G, C, H	5

Earliest Start takes E first because it starts at time 0 (it ties with F and wins by index), and then B. At time 13, D and G both start; D wins the tie by index, and because D runs until time 20 it blocks G, C, and H. The optimum takes the short job G instead, followed by C and H. This gives |EarliestStart(*I*)| = 3 and OPT(*I*) = 5. Figure 1 shows the instance as a timeline.

Figure 1. Timeline of the Earliest Start counterexample. Each bar covers [*s*, *f*). Dark bars are the jobs Earliest Start selects, and the check marks show each schedule.



Shortest Duration

Table 3. Shortest Duration counterexample (found by the Part 3 search after 178 attempts).

Input Intervals	Algorithm Selection	Greedy Size	Optimal Selection	OPT Size
A: (13, 18) B: (6, 13) C: (4, 8) D: (19, 23) E: (11, 19) F: (1, 5)	C, A, D	3	F, B, A, D	4

Shortest Duration takes C first because its duration of 4 is the shortest (it ties with D and F and wins by index). C overlaps both F and B, which are compatible with each other because F finishes at time 5 and B starts at time 6. The algorithm then adds D and A. This gives |ShortestDuration(*I*)| = 3 and OPT(*I*) = 4.

Longest Duration

Table 4. Longest Duration counterexample (found by the Part 3 search after 1 attempt).

Input Intervals	Algorithm Selection	Greedy Size	Optimal Selection	OPT Size
A: (9, 14) B: (2, 10) C: (14, 15) D: (2, 8) E: (18, 23) F: (11, 16)	B, F, E	3	D, A, C, E	4

Longest Duration takes B first because it is the longest job (length 8). D, the next longest, overlaps B. Of the jobs with length 5, A overlaps B, while E and F are selected, and C then overlaps F. The optimum skips B and takes D, A, C, and E. This gives |LongestDuration(I)| = 3 and OPT(I) = 4.

What the Counterexamples Prove

Finding a counterexample proves that an algorithm is not always optimal, because a single failing input is enough. Failing to find a counterexample does not prove that an algorithm is optimal. A search only covers the instances it tried, and a failure may exist outside them. Our searches and experiments found no counterexample for Earliest Finish, but that is only evidence. Its optimality follows from the exchange argument, not from the search.

Experimental Methodology

Each experiment is run from the A1 folder: python3 part-3/main.py for Part 3, python3 part-4/main.py for Part 4, python3 part-5/main.py for Part 5, and python3 part-6/main.py for Part 6, or from the menu started by python3 main.py. Every run resets its random seed, so the instances and results of Parts 3 to 5 reproduce exactly. Part 6 also uses the same inputs on every run, but its timings depend on the machine. The tables below are taken from the saved CSV files counterexamples.csv, part4_validation_results.csv, comparison_summary.csv, and part6_results.csv.

Part 3: Searching for Counterexamples

Part 3 first runs the two required sanity tests and stops if either one fails. It then searches for counterexamples with seed 4050. Each random instance has 4 to 10 jobs, with integer start times from 0 to 20 and integer durations from 1 to 8. For each heuristic, the search tries up to 5,000 instances and stops at the first one on which the heuristic selects fewer jobs than the exact solver. It found counterexamples for Earliest Start after 4 attempts, Shortest Duration after 178, and Longest Duration after 1. Part 3 then compares Earliest Finish with the exact solver on 100 more random instances with 1 to 10 jobs, and Earliest Finish matched on all of them.

Part 4: Validating Earliest Finish

Part 4 generates 100 instances with seed 2026. Each instance has 15 jobs, and the instances rotate evenly through four distributions, giving 25 instances of each type:

- Sparse: short durations (1 to 5), with starts spread over [100, 500].
- Heavily Overlapping: starts in [100, 120] and durations from 50 to 100.
- Varied Lengths: starts in [100, 300] and durations that are either short (1 to 3) or long (50 to 100).
- Clustered Starts: starts within 5 of the milestones 100, 200, or 300 and durations from 10 to 30.

Earliest Finish was compared with the exact solver on every instance (Table 5).

Table 5. Instances on which Earliest Finish matched the exact optimum (Part 4).

Distribution	Matched OPT	Match Rate
Sparse	25/25	100.0%
Heavily Overlapping	25/25	100.0%
Varied Lengths	25/25	100.0%
Clustered Starts	25/25	100.0%
Total	**100/100**	**100.0%**

Earliest Finish matched the exact optimum on all 100 instances. This agreement is evidence that the implementation is working, but it is not a proof that the algorithm is optimal.

Part 5: Comparing the Four Algorithms

Part 5 generates 120 instances with seed 4050, 30 for each of four types: sparse, overlapping, varied lengths, and clustered. Each instance has between 6 and 12 jobs so that the exact solver can be applied. For each algorithm H, the ratio $|H(I)| / \mathrm{OPT}(I)$ is computed, where a ratio of 1 means the algorithm was optimal. Table 6 reports the number and percentage of instances solved optimally, the average ratio, and the worst ratio observed. Table 7 gives the worst instance observed for each algorithm.

Table 6. Solution quality of the four algorithms relative to the exact solver (Part 5, 120 instances).

Algorithm	Optimal Instances	Optimal (%)	Average Ratio	Worst Ratio
Earliest Finish	120/120	100.0%	1.000	1.000
Earliest Start	85/120	70.8%	0.910	0.286
Shortest Duration	116/120	96.7%	0.990	0.500
Longest Duration	81/120	67.5%	0.886	0.286

Table 7. Worst instance observed for each algorithm (Part 5). Selections are listed in time order, with their size in parentheses.

Algorithm	Instance	Input Intervals	Selected	Optimal	Ratio
Earliest Finish	None	Optimal on all 120 instances	—	—	1.000
Earliest Start	39 (varied lengths)	(26, 28), (10, 13), (16, 46), (36, 38), (31, 33), (21, 24), (27, 57), (12, 17), (33, 35), (18, 19), (20, 50)	(10, 13), (16, 46) (2)	(10, 13), (18, 19), (21, 24), (26, 28), (31, 33), (33, 35), (36, 38) (7)	0.286
Shortest Duration	46 (overlapping)	(3, 15), (7, 20), (6, 14), (9, 22), (10, 19), (4, 16), (0, 9)	(6, 14) (1)	(0, 9), (9, 22) (2)	0.500
Longest Duration	39 (varied lengths)	(26, 28), (10, 13), (16, 46), (36, 38), (31, 33), (21, 24), (27, 57), (12, 17), (33, 35), (18, 19), (20, 50)	(10, 13), (16, 46) (2)	(10, 13), (18, 19), (21, 24), (26, 28), (31, 33), (33, 35), (36, 38) (7)	0.286

The results show which kinds of instances mislead each algorithm:

- Earliest Finish was optimal on all 120 instances.
- Earliest Start was optimal on 70.8% of the instances. It is usually reasonable because the early job is often short, and it fails when a single long interval that starts early covers many later jobs. In its worst case (instance 39, varied lengths), it selected (10, 13) and then the long interval (16, 46), which blocks the other six jobs of the optimal schedule.
- Shortest Duration was optimal on 96.7% of the instances. It fails when a short interval lies across the gap between two compatible intervals. In its worst case (instance 46, overlapping), it selected (6, 14), which overlaps both jobs of the optimal pair (0, 9) and (9, 22).
- Longest Duration was optimal on 67.5% of the instances, the lowest of the four. Long intervals usually overlap many others, so selecting one early blocks much of the schedule. Its worst case was the same instance as Earliest Start's, where it chose the 30-unit interval (16, 46) first.

Part 6: Runtime

Part 6 times each algorithm on random inputs of 100, 1,000, 10,000, and 100,000 jobs, with the generator seeded at 4050. Every algorithm runs on the same input at each size. Each algorithm gets one warm-up run followed by 10 timed runs, and the reported value is the median (Table 8).

Table 8. Median running time in milliseconds over 10 runs (Part 6).

n	Earliest Finish	Earliest Start	Shortest Duration	Longest Duration
100	0.158	0.148	0.136	0.127
1,000	1.524	1.415	1.577	1.486
10,000	20.132	18.777	21.684	20.454
100,000	258.351	242.894	283.845	275.576

Sorting appears in two places in every algorithm. The jobs are sorted by the algorithm's priority, and the endpoints are sorted once for the compatibility check. Each sort takes $O(n \log n)$ time, and the n Fenwick tree checks add another $O(n \log n)$, so the total is $O(n \log n)$.

When n increased tenfold, the median time grew by about 9.6 to 11.7 times from 100 to 1,000 jobs, by 13.2 to 13.8 times from 1,000 to 10,000 jobs, and by 12.8 to 13.5 times from 10,000 to 100,000 jobs. $O(n \log n)$ predicts about 15, 13, and 12.5 times for these three steps, so the measurements are consistent with it. Four input sizes cannot prove an asymptotic bound, however, and they cannot rule out other growth rates that are close to it.

Earliest Start was the fastest and Shortest Duration the slowest at every size from 1,000 up, but the largest gap is only about 17% (242.894 ms compared with 283.845 ms at 100,000 jobs). All four algorithms share the same helper, so these small gaps may come from machine noise and should not be read as one algorithm being faster.

Exact Solver

The exact solver in algorithms/optimal_algorithm.py is a brute-force search. It enumerates every subset of the n jobs in increasing size order using itertools.combinations. For each subset it checks every pair of jobs with the half-open overlap rule, so touching endpoints are allowed. A subset is kept when no pair overlaps and it is larger than the best subset found so far. Because every subset is examined, the solver always returns an optimal solution.

There are 2^n subsets. For a subset of size k, the pairwise check takes $O(k^2)$ time, so the total running time is $O(n^2 \cdot 2^n)$. The number of subsets doubles with each added job, so the solver is practical only for small n. We used it with 15 jobs in Part 4 and with 6 to 12 jobs in Part 5.

Engineering Reflection

1. Which incorrect greedy heuristic appeared most convincing before you tested it?

Before testing, Earliest Start seemed like the most convincing algorithm. "Start working as soon as possible" sounds like a sensible way to use time, and it was optimal on most instances (70.8%), but a long job at the start can ruin the schedule. Earliest Finish turned out to be the correct way to apply the "early" idea: finishing early, not starting early, is what leaves room for the jobs that follow.

2. What did the counterexample reveal about why the local rule fails?

The counterexamples show that each of the other three algorithms has a specific weak point. Earliest Start is misled by a long job that starts first, Shortest Duration by a short job that sits across two compatible jobs, and Longest Duration by long jobs that overlap many others. In each case the rule looks at a single property of one job and ignores how much of the rest of the schedule that job blocks. Each counterexample proves that its algorithm is not always optimal, but it says nothing about how often the algorithm fails. Part 5 measures that.

3. What did the exact solver contribute to the development process?

The exact solver gave us a ground truth. It tries every combination of jobs, so it is very inefficient, but it is guaranteed to return an optimal solution. It confirmed each counterexample and provides the OPT value that every ratio in Parts 4 and 5 is measured against. Without it we could only see what each algorithm selected, not whether the selection was the best possible.

4. What is the difference between experimentally validating an algorithm and proving it correct?

Experiments can only show that an algorithm agrees with the optimum on the instances that were tested. Earliest Finish agreed with the exact solver on 220 random instances. This is evidence that our implementation is correct, but the instances are small and random, so a failure on other sizes or structures would go unnoticed. A proof covers every input. The exchange argument proves that Earliest Finish is optimal: replacing the first job of any optimal schedule with the Earliest Finish job never reduces the number of jobs, and repeating this argument on the remaining jobs shows that the greedy schedule is optimal.

Conclusion

Earliest Finish matched the exact optimum on all 100 Part 4 instances and on all 120 Part 5 instances, and the exchange argument shows that it is optimal in general. Earliest Start, Shortest Duration, and Longest Duration were not always optimal. They matched the optimum on 70.8%, 96.7%, and 67.5% of the Part 5 instances. All four algorithms run in $O(n \log n)$ time, and the measured running times are consistent with this. The main limitation of the experiments is size: the exact solver restricts the comparison to small instances, so the results are evidence rather than proof.

Group Contribution Statement

Connor	Part 4, report, revisions
Chris	Part 6, part 7
Carson	Part 1, 2, overall structure
Alexis	Part 3, 5

AI Use Statement:

We used Claude and ChatGPT to review ideas, help generate code, and review overall submission.